

Name: Balaji Patil id no: 2520080065

Course: ossp section: 10 skill week 4

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Output

```
balaji@BalajiPatil:~$ nano shell_parser.c
balaji@BalajiPatil:~$ |
```

```
balaji@BalajiPatil: ~ x + v
GNU nano 7.2 shell_parser.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

typedef struct {
    char *tokens[MAX_TOKENS];
    int count;
} TokenList;

/* Tokenization */
void tokenize(char *input, TokenList *list) {
    list->count = 0;

    char *token = strtok(input, " \\t\\n");

    while (token != NULL && list->count < MAX_TOKENS - 1) {
        list->tokens[list->count++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    list->tokens[list->count] = NULL;
}

/* Validate tokens */
int validateTokens(TokenList *list) {
    if (list->count == 0) {
        printf("Error: Empty command\\n");
        return 0;
    }
    for (int i = 0; i < list->count; i++) {
        if (strcmp(list->tokens[i], "|") == 0 ||
            strcmp(list->tokens[i], ";") == 0) {

```

```
        if (i == 0 || i == list->count - 1) {
            printf("Syntax Error: Invalid delimiter position\n");
            return 0;
        }

        if (strcmp(list->tokens[i - 1], "|") == 0 ||
            strcmp(list->tokens[i + 1], ";") == 0) {
            printf("Syntax Error: Consecutive delimiters\n");
            return 0;
        }
    }

    return 1;
}

/* Display tokens for debugging */
void debugTokens(TokenList *list) {
    printf("\nTokens:\n");

    for (int i = 0; i < list->count; i++) {
        printf("Token[%d] = %s\n", i, list->tokens[i]);
    }
}

/* Execute command */
void executeCommand(TokenList *list) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        return;
    }
    if (pid == 0) {
        /* Child process */
        printf("Child: Executing command...\n");
    }
}
```

```

        execvp(list->tokens[0], list->tokens);

        perror("execvp failed");
        exit(1);
    }

    else {
        /* Parent process */
        printf("Parent: Waiting for child...\n");

        waitpid(pid, NULL, 0);

        printf("Parent: Child completed\n");
    }
}

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("\nmyShell> ");
        fflush(stdout);

        /* read() as shown in the flowchart */
        int n = read(STDIN_FILENO, input, MAX_INPUT - 1);

        if (n <= 0)
            break;

        input[n] = '\0';

        /* Remove newline */
        input[strcspn(input, "\n")] = '\0';

        /* Handle empty command */
        if (strlen(input) == 0) {

```

```
balaji@BalajiPatil: ~ × + v
GNU nano 7.2 shell_parser.c

        break;

    input[n] = '\0';

    /* Remove newline */
    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command. Please enter a command.\n");
        continue;
    }

    /* Exit command */
    if (strcmp(input, "exit") == 0)
        break;

    TokenList list;

    /* Tokenization */
    tokenize(input, &list);

    /* Debug parsing output */
    debugTokens(&list);

    /* Validation */
    if (!validateTokens(&list))
        continue;

    /* Execution */
    executeCommand(&list);
}
printf("\nShell terminated.\n");

    return 0;
}
```

```
balaji@BalajiPatil:~$ nano shell_parser.c
balaji@BalajiPatil:~$ gcc shell_parser.c -o shell_parser
balaji@BalajiPatil:~$ |
```

```
balaji@BalajiPatil:~$ nano shell_parser.c
balaji@BalajiPatil:~$ gcc shell_parser.c -o shell_parser
balaji@BalajiPatil:~$ ./shell_parser|
```

```
balaji@BalajiPatil:~$ nano shell_parser.c
balaji@BalajiPatil:~$ gcc shell_parser.c -o shell_parser
balaji@BalajiPatil:~$ ./shell_parser

myShell> ls
```

```
balaji@BalajiPatil:~$ nano shell_parser.c
balaji@BalajiPatil:~$ gcc shell_parser.c -o shell_parser
balaji@BalajiPatil:~$ ./shell_parser

myShell> ls

Tokens:
Token[0] = ls
Parent: Waiting for child...
Child: Executing command...
File1.c File1.txt File2.txt OS OSSP copy copy.c first05 nan nan.c shell shell.c shell_parser
shell_parser.c week4 week4.c week4.c.save
Parent: Child completed

myShell> █
```

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

Output

```
balaji@BalajiPatil: ~  
balaji@BalajiPatil:~$ nano task2.c  
balaji@BalajiPatil:~$
```

```
GNU nano 7.2 task2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
#define MAX_INPUT 1024  
#define MAX_ARGS 100  
  
typedef struct {  
    char *args[MAX_ARGS];  
    int count;  
} Command;  
  
/* Parse input and handle single quotes */  
int parseCommand(char *input, Command *cmd) {  
    int i = 0;  
    int arg = 0;  
  
    while (input[i] != '\0') {  
        /* Skip whitespace */  
        while (input[i] == ' ' || input[i] == '\t')  
            i++;  
  
        if (input[i] == '\0')  
            break;  
  
        char *buffer = malloc(MAX_INPUT);  
        int j = 0;  
  
        /* Single quoted string */  
        if (input[i] == '\'' ) {  
            i++; // Skip opening quote  
            while (input[i] != '\0' && input[i] != '\0') {  
                buffer[j++] = input[i++];  
            }  
        }  
    }  
}
```

```

        /* Check closing quote */
        if (input[i] != '\') {
            printf("\nSyntax Error: Unmatched single quote.\n");
            free(buffer);
            return 0;
        }

        i++; // Skip closing quote
    }

    /* Normal argument */
    else {
        while (input[i] != '\0' &&
               input[i] != ' ' &&
               input[i] != '\t') {

            if (input[i] == '\') {
                i++;

                while (input[i] != '\0' && input[i] != '\')
                    buffer[j++] = input[i++];

                if (input[i] != '\') {
                    printf("\nSyntax Error: Unmatched single quote.\n");
                    free(buffer);
                    return 0;
                }

                i++;
            }
            else {
                buffer[j++] = input[i++];
            }
        }
    }
}

```

```
    }

    buffer[j] = '\\0';

    cmd->args[arg++] = buffer;

    if (arg >= MAX_ARGS - 1)
        break;
}

cmd->args[arg] = NULL;
cmd->count = arg;

return 1;
}

/* Display parsing result */
void displayParsing(Command *cmd) {
    printf("\\nParsing Result:\\n");

    for (int i = 0; i < cmd->count; i++) {
        printf("Argument %d: %s\\n", i + 1, cmd->args[i]);
    }
}

/* Execute command */
void executeCommand(Command *cmd) {

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return;
    }

    if (pid == 0) {
```

```
        printf("\nChild PID: %d\n", getpid());

        execvp(cmd->args[0], cmd->args);

        perror("execvp failed");
        exit(1);
    }

    else {

        waitpid(pid, NULL, 0);

        printf("\nCommand execution completed.\n");
    }
}

/* Free memory */
void freeCommand(Command *cmd) {
    for (int i = 0; i < cmd->count; i++)
        free(cmd->args[i]);
}

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("\nmyShell> ");
        fflush(stdout);

        int n = read(STDIN_FILENO, input, MAX_INPUT - 1);

        if (n <= 0)
            break;
        input[n] = '\0';
    }
}
```

```
/* Empty command */
if (strlen(input) == 0) {
    printf("Empty command.\n");
    continue;
}

/* Exit */
if (strcmp(input, "exit") == 0)
    break;

Command cmd;

/* Parse */
if (!parseCommand(input, &cmd))
    continue;

/* Validate */
if (cmd.count == 0) {
    printf("Empty command.\n");
    continue;
}

/* Show parsing result */
displayParsing(&cmd);

/* Execute */
executeCommand(&cmd);

/* Free memory */
freeCommand(&cmd);
}

printf("\nShell terminated.\n");

return 0;
}
```

```
shivam@MSI123:~$ gcc task2.c -o task2
shivam@MSI123:~$ ./task2
```

```
myShell> echo Hello
```

```
Parsing Result:
Argument 1: echo
Argument 2: Hello
```

```
Child PID: 720
Hello
```

```
Command execution completed.
```

```
myShell> echo 'Hello World'
```

```
Parsing Result:
Argument 1: echo
Argument 2: Hello World
```

```
Child PID: 721
Hello World
```

```
Command execution completed.
```

```
myShell> echo 'Hello World
```

```
Syntax Error: Unmatched single quote.
```

```
myShell> echo $HOME
```

```
Parsing Result:
Argument 1: echo
Argument 2: $HOME
```

```
Child PID: 722
$HOME
```

```
Command execution completed.
```

```
myShell> |
```

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

Example Output:

Case1:

```
myShell> echo "Hello $USER"
```

--- Parsed Arguments ---

Argument 1: echo

Argument 2: Hello raghu

Child Process

PID = 3022

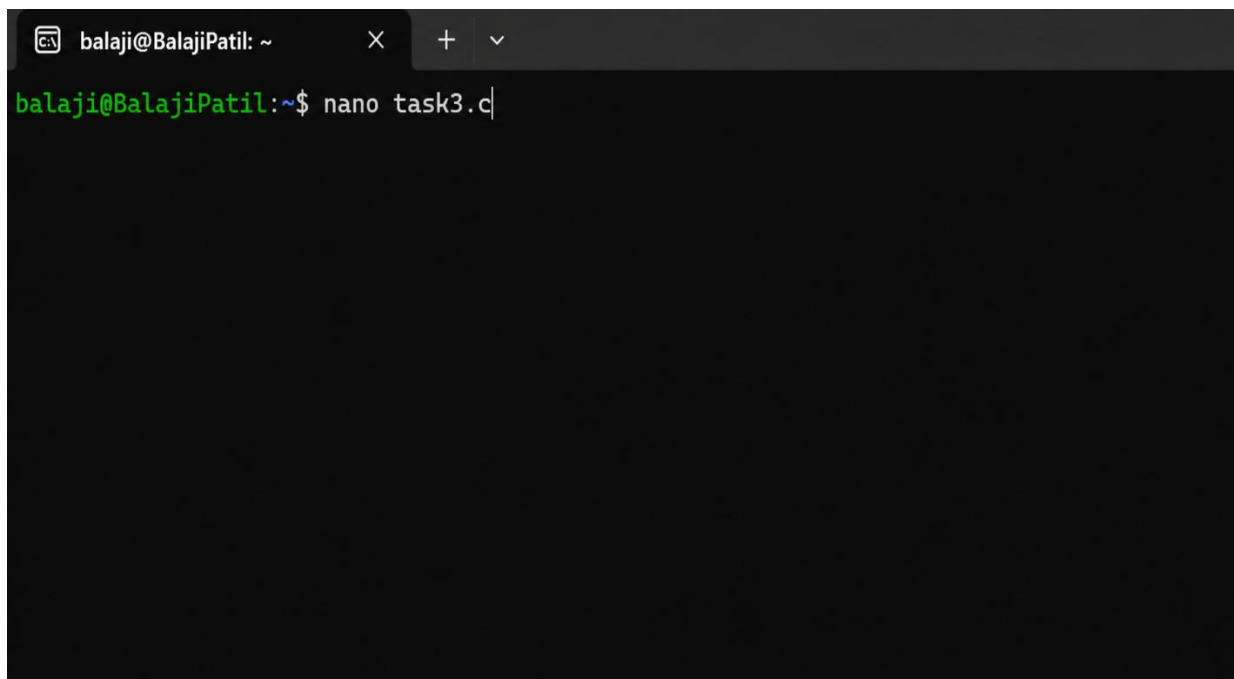
Hello raghu

Child process completed.

Case2:

```
myShell> echo "Hello World
```

Syntax Error: Unmatched double quote.

A terminal window with a dark background. The title bar shows 'balaji@BalajiPatil: ~' and window control buttons. The prompt is 'balaji@BalajiPatil:~\$' and the command 'nano task3.c' is entered. The terminal content is as follows:

```
balaji@BalajiPatil:~$ nano task3.c
```

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 20
#define MAX_LEN 1024

int main() {
    char input[MAX_LEN];

    while (1) {

        printf("myShell> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        /* Check unmatched double quote */
        int quoteCount = 0;
        for (int i = 0; input[i]; i++) {
            if (input[i] == '"')
                quoteCount++;
        }

        if (quoteCount % 2 != 0) {
            printf("Syntax Error: Unmatched double quote.\n");
            continue;
        }

        char *args[MAX_ARGS];
        int argc = 0;
```

```
int argc = 0;

char parsed[MAX_LEN];
int p = 0;

for (int i = 0; input[i]; ) {

    while (input[i] == ' ')
        i++;

    if (input[i] == '\\0')
        break;

    char token[256];
    int t = 0;

    /* Handle double quoted string */
    if (input[i] == '"') {
        i++;

        while (input[i] && input[i] != '"') {
            if (input[i] == '$') {
                i++;

                char var[100];
                int v = 0;
                while ((input[i] >= 'A' && input[i] <= 'Z') ||
                    (input[i] >= 'a' && input[i] <= 'z') ||
                    (input[i] == '_')) {
                    var[v++] = input[i++];
                }
            }
        }
    }
}
```

```
}

args[argc] = NULL;

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++) {
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid == 0) {

    printf("\nChild Process\n");
    printf("PID = %d\n\n", getpid());

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

else {

    waitpid(pid, NULL, 0);
    printf("\nChild process completed.\n");
}

for (int i = 0; i < argc; i++)
    free(args[i]);

return 0;
}
```

```
balaji@BalajiPatil:~$ nano task3.c
balaji@BalajiPatil:~$ gcc task3.c -o task3 |
```

```
balaji@BalajiPatil:~$ nano task3.c
balaji@BalajiPatil:~$ gcc task3.c -o task3
balaji@BalajiPatil:~$ ./task3
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello balaji

Child Process
PID = 7854

Hello balaji

Child process completed.

myShell> echo "Hello World"
Syntax Error: Unmatched double quote.
myShell> |
```